Chuleta de Spring Boot · UT4 y UT5

1 · El proyecto

Starter	Para qué
`spring-boot-starter-web`	`@RestController`, Tomcat, JSON
`spring-boot-starter-validation`	`@Valid`, `@NotBlank`…
`spring-boot-starter-data-jpa`	**UT5**: entidades, `JpaRepository`, `@Transactional`
`spring-boot-starter-jdbc`	**UT4**: `JdbcClient`, SQL a mano
`h2` · `postgresql`	Las bases de datos *(scope `runtime`)*
`lombok` · `spring-boot-devtools`	Menos código · reinicio automático

```
es/iesx/app/
├── App.java                          ← en el paquete RAÍZ
(@ComponentScan baja desde aquí)
├── common/GlobalExceptionHandler.java
└── funkos/                           ← un paquete POR
RECURSO, y dentro, por capa
    ├── models/  repositories/  dto/  mappers/
    └── exceptions/  services/ (interfaz + Impl)  controllers/
```

```
mvn spring-boot:run                          # o ./mvnw
mvn clean package && java -jar target/app.jar
SPRING_PROFILES_ACTIVE=postgres mvn spring-boot:run
```

2 · Configuración

```
server.port=${PORT:8080}                              # variable
de entorno, o 8080
api.path=/api
api.version=v1
spring.profiles.active=${SPRING_PROFILES_ACTIVE:dev}

spring.datasource.url=jdbc:h2:file:./datos/
app;AUTO_SERVER=TRUE   # o jdbc:h2:mem:app
spring.datasource.username=sa
spring.h2.console.enabled=true                        # /h2-
console · false en prod

spring.jpa.hibernate.ddl-auto=update                  # UT5 · ver
tabla
spring.jpa.show-sql=true
spring.jpa.open-in-view=false                         # ¡apágalo!
spring.jpa.defer-datasource-initialization=true   # data.sql
DESPUÉS de crear tablas
spring.sql.init.mode=always
server.error.include-message=always                   # never en
prod
```

`ddl-auto`	Qué hace	Dónde
`update`	Añade lo que falta, nunca borra	**dev**
`create`	Borra y crea al arrancar	examen, H2 en memoria
`validate`	Si no cuadra, **no arranca**	**prod**

`data.sql`	H2	PostgreSQL
Sin duplicar	`MERGE … KEY(id)`	`INSERT … ON CONFLICT (id) DO NOTHING`
Recolocar el id	`ALTER TABLE t ALTER COLUMN id RESTART WITH 7`	`SELECT setval('t_id_seq', …)`

Prioridad: línea de comandos › variable de entorno (`SERVER_PORT`) › `application-{perfil}` › `application.properties`

3 · Componentes, inyección y Lombok

Anotación	Capa
`@RestController`	Web
`@Service`	Negocio
`@Repository`	Datos (en JPA no hace falta: la interfaz basta)
`@Component`	Mappers y utilidades
`@Configuration` + `@Bean`	Objetos de clases que **no son tuyas**

```
@Service
public class FunkosServiceImpl implements FunkosService {
    private final FunkosRepository
repositorio;                    // final
    public FunkosServiceImpl(FunkosRepository repositorio)
{       // por constructor
        this.repositorio = repositorio;
    }
}
```

Lombok	Genera
`@Getter` `@Setter`	Getters y setters (`isX()` en `boolean`)
`@NoArgsConstructor`	Constructor vacío: **JPA lo necesita**
`@RequiredArgsConstructor`	Constructor con los campos `final`
`@AllArgsConstructor` · `@ToString`	Todos los campos · `toString`

AVISO Nunca `@Data` en una entidad: su `equals`/`hashCode` y `toString` recorren las relaciones (bucles y N+1). **AVISO** Los beans son **singleton**: un servicio **no guarda estado** en campos.

4 · El controlador

```
@RestController
@RequestMapping("${api.path}/${api.version}/funkos")
public class FunkosRestController {

    @GetMapping                                          //
GET  /api/v1/funkos
    public ResponseEntity<Page<FunkoResponse>> findAll(
            @RequestParam Optional<String>
nombre,          // ?nombre=goku  (opcional)
            @PageableDefault(size = 20, sort = "id") Pageable
pageable) {
        return ResponseEntity.ok(servicio.findAll(nombre,
sanear(pageable)));
    }

    @GetMapping("/{id}")                                 //
GET /api/v1/funkos/1
    public ResponseEntity<FunkoResponse>
findById(@PathVariable Long id) { … }

    @PostMapping
    public ResponseEntity<FunkoResponse> save(@Valid
@RequestBody FunkoCreateRequest dto) {
        var creado = servicio.save(dto);
        return ResponseEntity.created(URI.create("/api/v1/
funkos/" + creado.id())).body(creado);
    }

    @PutMapping("/{id}")    // reemplaza TODO      →
ResponseEntity.ok(...)
    @PatchMapping("/{id}")  // solo lo que llega   →
ResponseEntity.ok(...)
    @DeleteMapping("/{id}") // → servicio.deleteById(id);
return ResponseEntity.noContent().build();
}
```

	Éxito	No existe	Datos malos	Choca con una regla
GET lista	200 (vacía también)	—	—	—
GET uno	200	**404**	—	—
POST	**201 +** `Location`	—	**400**	**409**
PUT / PATCH	200	404	400	409
DELETE	**204**	404	—	409

AVISO El controlador **no tiene lógica**: recibe, llama al servicio y devuelve el código. **Nunca devuelve la entidad.** **AVISO** Identifica el recurso → ruta (`/{id}`). Filtra, ordena o pagina → query (`?precioMax=20`).

5 · DTO y validación

```
public record FunkoResponse(Long id, String nombre, BigDecimal
precio, String categoria) {}

public record FunkoCreateRequest(                        //
POST y PUT: sin id ni fechas
        @NotBlank(message = "El nombre es obligatorio")
@Size(max = 100) String nombre,
        @NotNull @Positive BigDecimal precio,
        @NotNull @PositiveOrZero Integer cantidad,
        @NotBlank @Pattern(regexp = "97[89]-\\d{1,5}-\\d{1,7}-
\\d{1,7}-\\d") String isbn,
        @NotNull Long categoriaId) {}

public record FunkoUpdateRequest(                        //
PATCH: SIN @NotNull/@NotBlank
        @Size(max = 100) String nombre,
        @Positive BigDecimal precio) {}
```

Anotación	Para
`@NotNull`	Números, fechas, ids, booleanos
`@NotBlank`	`String`: no nulo y no solo espacios
`@NotEmpty`	Colecciones
`@Size(min=, max=)`	Longitud de texto o tamaño de lista
`@Min` `@Max`	Enteros
`@Positive` `@PositiveOrZero` `@DecimalMin("0.0")` `@Digits(integer=, fraction=)`	Números y `BigDecimal`
`@Email` `@Pattern(regexp = "…")`	Texto
`@Past` `@PastOrPresent` `@Future`	Fechas
`@Valid`	En el controlador, **y** en listas anidadas: `@NotEmpty` `@Valid` `List<LineaRequest>`

AVISO **Sin `@Valid` en el parámetro no se valida nada**, y no avisa.

Nivel	Dónde	Código
Formato de un campo	Anotaciones del DTO	400
Regla de negocio (consulta la BD)	**Servicio**	409 / 400
Integridad (`UNIQUE`, `FK`)	Tabla	409 vía handler

6 · Excepciones

```
@ResponseStatus(HttpStatus.NOT_FOUND)                    //
404 · CONFLICT 409 · BAD_REQUEST 400
public class FunkoNotFoundException extends RuntimeException {
    public FunkoNotFoundException(Long id) { super("No existe
el funko con id: " + id); }
}

// en el servicio, SIN hablar de HTTP:
repositorio.findById(id).orElseThrow(() -> new
FunkoNotFoundException(id));
```

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public Map<String, Object>
validacion(MethodArgumentNotValidException ex) {
        Map<String, String> campos = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
          .forEach(e -> campos.put(e.getField(),
e.getDefaultMessage()));
        return Map.of("status", 400, "campos", campos);
    }
    // mismo patrón: HttpMessageNotReadableException (JSON
roto) → 400
    //   MethodArgumentTypeMismatchException (/funkos/abc) →
400
    //   DataIntegrityViolationException (UNIQUE, FK) → 409
    //   PropertyReferenceException (?sort=inventado) → 400
}
```

AVISO `PropertyReferenceException`: `org.springframework.data.mapping` en **Boot 3** · `org.springframework.data.core` en **Boot 4**. **AVISO** Tus excepciones con `@ResponseStatus` **no necesitan handler**.

7 · El servicio y `@Transactional`

```
import
org.springframework.transaction.annotation.Transactional;   //
 (NO jakarta.transaction)

@Override @Transactional(readOnly = true)               //
lecturas
public FunkoResponse findById(Long id) {
    return mapper.toResponse(buscar(id));
}

@Override @Transactional                                //
escrituras
public FunkoResponse update(Long id, FunkoCreateRequest r) {
    var funko = buscar(id);                             // 1 ·
buscar (404)
    if (!
funko.getNombre().equalsIgnoreCase(r.nombre())        // 2 ·
reglas (409)
            &&
repositorio.existsByNombreIgnoreCase(r.nombre()))
        throw new FunkoConflictException("Ya existe " +
r.nombre());
    mapper.actualizar(funko, r,
buscarCategoria(r.categoriaId()));   // 3 · mapear
    return mapper.toResponse(funko);                    // 4 ·
SIN save(): dirty checking
}
```

AVISO Solo hace rollback con `RuntimeException`, y si la excepción **sale** del método. **AVISO** Funciona por proxy: `this.otroMetodo()` no abre transacción (igual que `@Cacheable`). **AVISO** Sin `@Transactional`, leer una relación `LAZY` → `LazyInitializationException`.

8 · Entidad JPA (UT5)

```
@Entity
@Table(name = "funkos")
@Getter @Setter @NoArgsConstructor
public class Funko {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String nombre;

    @Column(nullable = false, precision = 10, scale =
2)       // dinero: BigDecimal
    private BigDecimal precio;

    @Enumerated(EnumType.STRING) @Column(length =
20)          // NUNCA ORDINAL
    private Estado estado;

    @ManyToOne(fetch = FetchType.LAZY, optional =
false)       // lado de la FK
    @JoinColumn(name = "categoria_id", nullable = false)
    private Categoria categoria;

    @Column(name = "is_deleted", nullable = false)
    private boolean deleted = false;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt = LocalDateTime.now();

    @Column(name = "updated_at")
    private LocalDateTime updatedAt = LocalDateTime.now();

    @PreUpdate
    void alActualizar() { this.updatedAt =
LocalDateTime.now(); }
}
```

AVISO Una entidad **no puede ser** `record`: JPA necesita constructor vacío y setters. **AVISO** `@ManyToOne` es `EAGER` **por defecto** → ponle siempre `LAZY`. **AVISO** El lado inverso (`@OneToMany(mappedBy = "categoria")`) **no hace falta**: para contar, pregunta al repositorio del otro. **AVISO** Dos relaciones = **dos** `@ManyToOne`. Una N:M con datos propios = **dos 1:N** y una entidad en medio.

9 · Repositorio JPA (UT5)

```
public interface FunkosRepository
        extends JpaRepository<Funko, Long>,
JpaSpecificationExecutor<Funko> {

    Optional<Funko> findByIdAndDeletedFalse(Long id);
    boolean existsByNombreIgnoreCase(String nombre);
    long countByCategoriaIdAndDeletedFalse(Long categoriaId);
    List<Funko> findTop5ByDeletedFalseOrderByPrecioDesc();

    @Override                                           // el
buscador, SIN N+1
    @EntityGraph(attributePaths = {"categoria"})
    Page<Funko> findAll(Specification<Funko> spec, Pageable
pageable);

    @Query("SELECT f FROM Funko f WHERE f.categoria.nombre
= :cat AND f.cantidad > 0")
    List<Funko> conStock(@Param("cat") String cat);   // JPQL:
entidades y campos Java

    @Modifying(clearAutomatically = true, flushAutomatically =
true)
    @Query("UPDATE Funko f SET f.deleted = true WHERE f.id
= :id")
    int borradoLogico(@Param("id") Long id);          //
llámalo el ÚLTIMO, tras los setters
}
```

Ya vienen hechos: `findAll()` · `findAll(Pageable)` · `findById` → `Optional` · `save` · `deleteById` · `existsById` · `count`

En el nombre	SQL
`findBy` `existsBy` `countBy`	`SELECT` `EXISTS` `COUNT`
`And` `Or` `Not`	
`LessThan(Equal)` `GreaterThan(Equal)` `Between` `Before` `After`	comparaciones
`Containing` `StartingWith` `IgnoreCase`	`LIKE '%x%'` · `UPPER`
`True` `False` `IsNull` `In`	
`OrderByPrecioDesc` · `Top5`	`ORDER BY` · `LIMIT`

AVISO Parámetros siempre `:nombre` **con** `@Param`, nunca `?1` ni concatenando. **AVISO** Un campo mal escrito en el nombre → falla **al arrancar** (`PropertyReferenceException`).

10 · Paginación y ordenación

`?page=0&size=20&sort=precio,desc&sort=nombre,asc` · `page` **empieza en 0**

```
private Pageable sanear(Pageable p) {                    //
lista BLANCA + tope + desempate
    var permitidos = Set.of("id", "nombre", "precio");
    var orden = p.getSort().stream()
            .filter(o ->
permitidos.contains(o.getProperty())).toList();
    return PageRequest.of(p.getPageNumber(),
Math.min(p.getPageSize(), 100),
            orden.isEmpty() ? Sort.by("id") :
Sort.by(orden).and(Sort.by("id")));
}
```

```
Pageable p = PageRequest.of(0, 20,
Sort.by("precio").descending());   // a mano
Page<FunkoResponse> r = repositorio.findAll(spec,
p).map(mapper::toResponse);
```

`Page` → `getContent()` `getTotalElements()` `getTotalPages()` `getNumber()` `hasNext()` `map(fn)` JSON: `{ "content": [...], "totalElements": 6, "totalPages": 3, "number": 0, "size": 2, ... }`

AVISO Página fuera de rango = **200 con** `content: []`, no 404. `?sort=inventado` nunca puede dar 500.

11 · `Specification`: filtros opcionales (UT5)

```
Specification<Funko> spec = (root, query, cb) ->
cb.isFalse(root.get("deleted"));   // base

if (nombre.isPresent()) {
    spec = spec.and((root, query, cb) ->
            cb.like(cb.lower(root.get("nombre")), "%" +
nombre.get().toLowerCase() + "%"));
}
if (categoria.isPresent()) {                                //
navega la relación: JOIN solo
    spec = spec.and((root, query, cb) ->

cb.equal(cb.upper(root.get("categoria").get("nombre")),
categoria.get().toUpperCase()));
}
if (precioMax.isPresent()) {
    spec = spec.and((root, query, cb) ->
cb.lessThanOrEqualTo(root.get("precio"), precioMax.get()));
}
return repositorio.findAll(spec,
pageable).map(mapper::toResponse);
```

Quieres	Condición
`=`	`cb.equal(root.get("campo"), v)`
`LIKE`	`cb.like(cb.lower(root.get("campo")), "%" + v.toLowerCase() + "%")`
`<=` `>=` `>`	`cb.lessThanOrEqualTo` · `cb.greaterThanOrEqualTo` · `cb.greaterThan`
`BETWEEN` · `IS NULL` · `= true`	`cb.between(...)` · `cb.isNull(...)` · `cb.isTrue(...)`

AVISO `spec = spec.and(...)`, **no** `spec = ...`: si reasignas, el último filtro borra a los demás. **AVISO** Nombres **de la entidad** (`createdAt`), no de la tabla (`created_at`).

12 · JDBC con `JdbcClient` (UT4)

```
@Repository
public class FunkosRepositoryImpl implements FunkosRepository
{
    private final JdbcClient jdbc;  private final
FunkoRowMapper rowMapper;

    jdbc.sql("SELECT * FROM funkos ORDER BY
id").query(rowMapper).list();
    jdbc.sql("SELECT * FROM funkos WHERE id
= :id").param("id", id).query(rowMapper).optional();
    jdbc.sql("SELECT COUNT(*) FROM funkos WHERE nombre
= :n").param("n", n).query(Integer.class).single();

    var keys = new
GeneratedKeyHolder();                          // INSERT con id
generado
    jdbc.sql("INSERT INTO funkos (nombre, precio) VALUES
(:nombre, :precio)")
        .param("nombre", f.nombre()).param("precio",
f.precio())
        .update(keys,
"ID");                                     // columna en
MAYÚSCULAS
    Long id = ((Number) keys.getKeys().get("ID")).longValue();

    int filas = jdbc.sql("DELETE FROM funkos WHERE id
= :id").param("id", id).update();  // 0 → 404
}
```

```
@Component
public class FunkoRowMapper implements RowMapper<Funko> {
    public Funko mapRow(ResultSet rs, int n) throws
SQLException {
        return new Funko(rs.getLong("id"),
rs.getString("nombre"), rs.getBigDecimal("precio"),

rs.getTimestamp("created_at").toLocalDateTime());
    }
}
```

Filtros: `new StringBuilder("SELECT * FROM funkos WHERE 1 = 1")` + `sql.append(" AND precio <= :max")` + `params.put("max", v)` → `.params(params)`. **Stock sin condición de carrera:** `UPDATE p SET stock = stock - :n WHERE id = :id AND stock >= :n` → 0 filas = no había.

AVISO Nunca `" AND nombre = '" + n + "'"`: inyección SQL. **AVISO** H2 en fichero: `CREATE TABLE IF NOT EXISTS` y `MERGE … KEY(id)`, porque `schema.sql` se ejecuta en cada arranque.

13 · Java: colecciones, `Optional` y streams

```
List<String> l = new ArrayList<>(List.of("a", "b"));   //
List.of(...) es INMUTABLE
l.add("c"); l.remove("a"); l.contains("b"); l.size();
l.isEmpty(); l.get(0);
Set<String> s = new HashSet<>();                          // sin
repetidos · TreeSet: ordenado
Map<String, Integer> m = new HashMap<>();                 //
LinkedHashMap: orden de inserción
m.put("k", 1); m.get("k"); m.getOrDefault("x", 0);
m.containsKey("k");
m.merge("k", 1, Integer::sum);                            //
contador: suma 1
m.computeIfAbsent("k", k -> new ArrayList<>()).add(x);  //
agrupar a mano
for (var e : m.entrySet()) { e.getKey(); e.getValue(); }
```

```
Optional<Funko> o = repositorio.findById(id);
o.orElseThrow(() -> new FunkoNotFoundException(id));   // el
más usado
o.map(mapper::toResponse);  o.isPresent();  o.ifPresent(f ->
…);  o.orElse(null);
```

```
lista.stream()
     .filter(f -> f.getPrecio().compareTo(BigDecimal.TEN) >
0)   // quedarse con
     .map(mapper::toResponse)
transformar
     .sorted(Comparator.comparing(FunkoResponse::precio).reverse(
     .limit(5)
     .toList();
List inmutable

stream.count();  stream.anyMatch(f -> f.isDeleted());
stream.findFirst();
stream.mapToInt(Linea::cantidad).sum();
stream.mapToDouble(Nota::getValor).average().orElse(0);
stream.map(Linea::subtotal).reduce(BigDecimal.ZERO,
BigDecimal::add);   // sumar BigDecimal

Collectors.groupingBy(Funko::getCategoria)
Map<Cat, List<Funko>>
Collectors.groupingBy(Funko::getCategoria,
Collectors.counting()) // Map<Cat, Long>
Collectors.toMap(Funko::getId,
Funko::getNombre)                 // Map<Long, String>
Collectors.joining(",
")                                        // String
```

```
BigDecimal a = new BigDecimal("12.50");               // con
String, NUNCA new BigDecimal(12.5)
a.add(b); a.subtract(b); a.multiply(BigDecimal.valueOf(3));
a.compareTo(b) > 0;   // no uses equals
a.setScale(2, RoundingMode.HALF_UP);
LocalDate.now(); LocalDate.of(2026, 1, 20); f.isBefore(otra);
f.plusDays(7);
LocalDateTime.now(); ChronoUnit.DAYS.between(a, b);
"Hola".toUpperCase(); s.trim(); s.isBlank(); s.contains("x");
s.equalsIgnoreCase(t);
"%s tiene %d".formatted(nombre, n);
```

AVISO Filtrar o contar con un stream **lo que puede hacer la base de datos** es el error: trae la tabla entera. Usa `countBy…`, `Specification` o `@Query`.

14 · Caché (UT4)

```
@EnableCaching                                  // en la
clase principal; sin esto no hace nada
@CacheConfig(cacheNames = "funkos")             // en el
servicio
@Cacheable(key = "#id")          public FunkoResponse
findById(Long id) { … }
@CacheEvict(allEntries = true)   public FunkoResponse
save(...)          { … }   // TODAS las escrituras
```

15 · Probar: Bruno y `curl`

```
meta {
  name: 04 Crear funko
  type: http
  seq: 4
}

post {
  url: {{baseUrl}}/api/v1/funkos
  body: json
  auth: none
}

body:json {
  { "nombre": "Goku", "precio": 22.5, "categoriaId": 1 }
}

assert {
  res.status: eq 201
  res.body.nombre: eq Goku
  res.headers.location: isDefined
}
```

En un listado paginado: `res.body.content.length: eq 3` `bruno.json` en la raíz · `environments/Local.bru` → `vars { baseUrl: http://localhost:8080 }` Operadores: `eq` `neq` `gt` `gte` `lt` `lte` `isDefined` `isNumber` `contains`

```
curl -s "localhost:8080/api/v1/funkos?
page=0&size=3&sort=precio,desc" | jq
curl -i -X POST localhost:8080/api/v1/funkos -H "Content-Type:
application/json" \
     -d '{"nombre":"Goku","precio":22.5,"categoriaId":1}'
curl -i -X DELETE localhost:8080/api/v1/funkos/1
```

Los veinte errores del examen

	Síntoma	Causa
1	201 con datos basura	Falta `@Valid` en el controlador
2	Las líneas de un pedido no se validan	Falta `@Valid` en la lista anidada
3	El PATCH da 400 si no mandas todo	`@NotNull`/`@NotBlank` en el DTO de actualizar
4	500 en vez de 404	`findById(id).get()` en lugar de `orElseThrow`
5	La excepción propia da 500	Le falta `@ResponseStatus`
6	200 en el POST	`ok(...)` en lugar de `created(URI)`
7	`LazyInitializationException`	Falta `@Transactional`, o se devuelve la entidad
8	`readOnly` no compila	`@Transactional` de `jakarta.transaction`
9	21 consultas para 20 filas	Falta el `@EntityGraph`
10	`Table "X" not found` al arrancar	Falta `defer-datasource-initialization=true`
11	No arranca: `Not a managed type`	Entidad sin `@Entity`, o en otro paquete
12	`Could not determine … JdbcType for Autor`	Relación sin `@ManyToOne`
13	El borrado sigue saliendo en el listado	`Specification` sin la base `deleted = false`
14	Los filtros no se suman	`spec = …` en vez de `spec = spec.and(…)`
15	`?sort=inventado` → 500	No se llama a `sanear`
16	PUT con su propio nombre da 409	La regla de unicidad no se excluye a sí misma
17	Los setters no se guardan	El `@Modifying` va **antes** de los setters
18	`TransactionRequiredException`	`@Modifying` sin `@Transactional`
19	415 en el POST	Falta `Content-Type: application/json`
20	El primer POST choca con un id	Falta `RESTART WITH` / `setval` tras insertar con ids fijos